Two Guesses, One Score: Coverage-Optimal Selection and Its Empirical Limits in ARC-AGI-2 Solver Pools

Theory, a pre-registered benchmark on real solver candidate pools, and a measured negative result

ARC Prize 2026 Paper Track team

Preprint — not peer reviewed · 18 October 2026

Working draft v4 — 2026-10-21 (Theorem 5 applied; pre-submission-score). NOT FROZEN. Base v4 sha256 `68eabd59…`; Theorem 5 patch applied after dry-run verification. v4 PDF to be rebuilt at freeze (`paper/paper.pdf` is still v3). Derived from v3 (sha256 `00435c14…29972`; v3 remains the published draft). This copy marks remaining pending slots for Figure 8/Table 11 (final LB snapshot), the S5b score, and the processed-task count; the first submission attempt (57456856) errored and its retry scored 0.00. Do not publish until every `V4-PEND` is resolved and the Lead freezes the final numbers. **No priority claim.**

Abstract

ARC-AGI-2 gives each test input exactly two attempts, scored by exact match, so choosing answers is a decision problem. We formalize it as coverage optimization — maximize U(a1,a2)=1−P(¬C(a1)∧¬C(a2)) — and prove five results: independent-case top-2 optimality, duplicate dominance, argmax-once suboptimality under correlated errors, an exact diversification threshold for when the second attempt pays, and a budget-allocation theorem. Simulation is positive (coverage 41.67% vs duplicate 16.67% and argmax 23.33%; +18.33 pp, 95% CI [11.67, 25.0]), but on real public pools the solver's own score ordering already saturates recoverable coverage: on 400 CompressARC tasks top-2 covers 20.00% and coverage-optimal 17.75% against 34.25% oracle, leaving 14.25 points that re-ranking cannot reach; a 16-input ARC-AGI-2 pilot ties every rule at the 31.25% oracle ceiling. Two attempts are not two chances unless the second candidate's correctness can be estimated independently of the first; the bottleneck is candidate generation/coverage. Six submissions scored: S5 **16.25%** (ref **57681632**, champion), S6 and S5_ens 15.42%, S1 and S5+LegC 14.17%, CPU probe 0.00%; the same pipeline measures **20.0% task-level** on a 20-task public-eval subset. No score exceeds 16.25; a reference replication reached 3/20 on dev20, far below the ~30 band. LB context: the 2026-10-20 Kaggle public-leaderboard snapshot (`analysis/results/leaderboard_snapshot_2026-10-20.json`; rank-percentile points, e.g. top-10% = rank 309 at 31.81; no medal-cutoff claim). No state-of-the-art claim is made.

---

1. Introduction

ARC-AGI-2 measures skill-acquisition efficiency on unseen grid transformations [1, 2]. Its Kaggle incarnation is efficiency-constrained [12]: the notebook re-runs offline for ~12 h without internet, and each input receives exactly two guesses. Because exact matches decide tasks, **selection** is a first-class decision problem separate from generation: a solver with the right grid in its pool still scores zero if its two attempts miss it.

Public practice ranks candidates by inverse consensus or a likelihood variant [3, 4, 8, 9] and submits the top two distinct grids. We re-derive selection from the scoring rule. Three observations drive the paper: what matters is P(either guess correct), not P(first); correlation is structural because candidates share TTT state, views, beam prefix and model; and a better rule needs calibrated probabilities and correlations.

We contribute a formalization, a pre-registered test, a real attempt, and a measured answer. (i) We define the coverage objective, prove five results including a diversification threshold and a budget-allocation theorem, and give an exact Bernoulli-correlation pair formula and calibrated estimator (§3). (ii) We

freeze a claim/downgrade protocol and benchmark real third-party pools at two scales plus simulated stress tests (§5). (iii) The measured result is bounded negative: the verdict is `DOWNGRADE_TO_ANALYSIS`, the simulation strongly positive (+18.33 pp over argmax, 95% CI [11.67, 25.0]), but on 400 CompressARC tasks top-2 covers 20.00% while coverage-optimal covers 17.75% against 34.25% oracle — 14.25 points not re-ranking-reachable; selection is not the bottleneck, candidate coverage/generation is. (iv) We report a real TTT submission attempt and GPU measurements (§4), data/leakage audit (§6), open-source system (§7), and landscape analysis (§8).

Nearest prior work: public-notebook heuristics include coverage-aware set selection, MDL-divergence two-slot selection and structural attempt-2 distinctness [17, 23], and rank-based top-2 with minority recoveries [30]. A concurrent instrumentation study of a faithful NVARC reproduction — empirical, not a coverage model — reports a comparable pattern (26.1% vs 32.5% perfect selection; a grouped CV reranker no better than the deployed vote rule) [26]; the headroom observation is already documented. A coverage-vs-selection identifiability gap is formalised for single-answer deployment [28], and on ARC-AGI-1 an unbiased pass@k decomposition finds that selection already recovers ≈95% of the candidate ceiling (generation-bound) [27]. The coverage objective is classical [24, 25, 29]; we provide the two-attempt formalization, calibrated/correlated estimator, and score-sufficiency criterion.

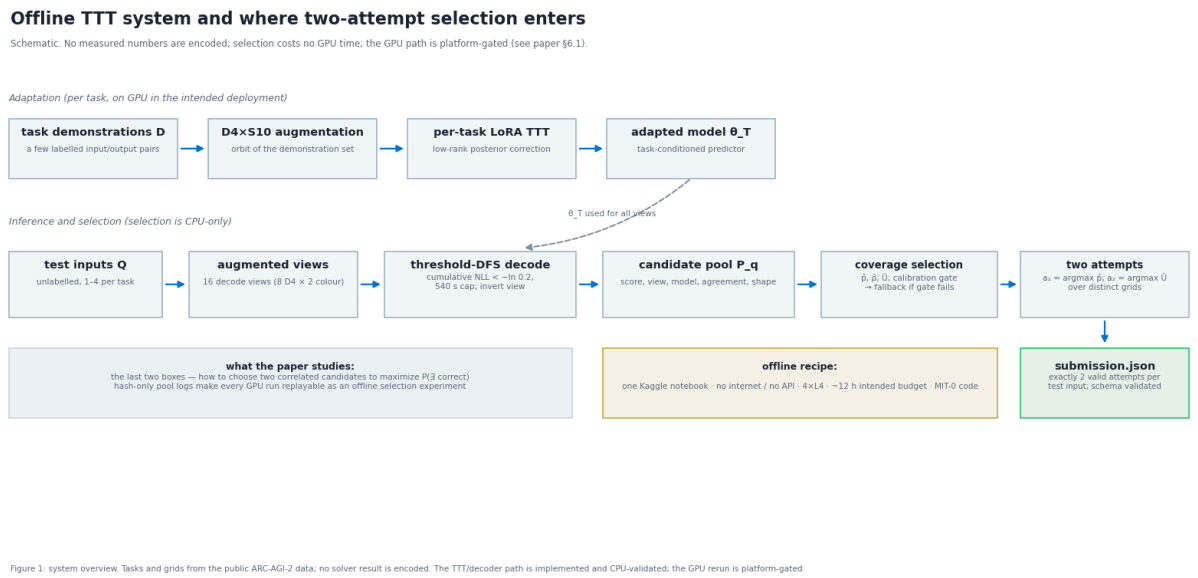


Figure 1. Offline TTT system and where selection enters: demonstrations ⟶ augmented TTT ⟶ threshold-DFS decoding ⟶ candidate pool with scores and view provenance ⟶ calibrated coverage selection ⟶ two-attempt submission. Selection costs no GPU time. Schematic; no measured numbers encoded.

2. Problem formalization and notation

Task and submission. A task is T=(D,Q) with labelled demonstrations D and unlabelled test inputs Q=(q_1,…,q_m); grids are integer arrays over colours 0–9, at most 30×30. For each test input the submission provides an ordered pair of distinct grids (a_1,a_2); C(a)∈{0,1} is the latent event that a is exactly required. The two metric definitions — per-input coverage and task-level all-or-nothing — are both monotone in P(C(a_1)∨C(a_2)); the per-input score averages it, the task-level score is its m-fold product [3, 21], so the objective below is valid under either.

Coverage objective. For attempts a_1,a_2,

$$U(a_1,a_2) := 1 - P(\neg C(a_1) \wedge \neg C(a_2)). \quad (1)$$

Maximizing U differs from maximizing P(C(a_1)): the joint failure term couples the attempts.

Candidate pools from TTT. TTT adapts a pretrained ARC-SFT model θ on augmented demonstrations and decodes under transformed views; for view v (augmentation g_v, beam path, model member) the predicted grid is inverted as ŷ_v=g_v^{-1}(f_θ(g_v·q)). The pool for test input q is

P_q = {(y_i, s_i, view_i, model_i, agree_i, shape_i)}, (2)

with s_i a plausibility score, agree_i the view count, and shape_i a demonstration-derived flag. The pool's oracle ceiling bounds any rule's per-input accuracy.

Relation classes. Pairs are grouped by shared failure factors: `same_grid` (correlation 1, never worth two attempts), `same_view` (same TTT state/augmentation), `same_model` (different views), and `cross_model`/`cross_source` (lowest correlation). The rule needs per-class error correlations, observable in metadata without labels.

id	source	benchmark	scale / unit	published anchor	role
B	`vanductan/arc-agi-2-candidate-dump` (displayed LB 30.97)	ARC-AGI-2 public eval	20 tasks / 29 test inputs; unit = input; 347 candidate files	oracle-any-2 9/29 = 31.0%; source top-2 8/29 = 27.6%	primary, **pilot** (CIs mandatory)
A	CompressARC [6] bundled histories (Liao & Gu 2025; MIT)	ARC-AGI-1 train/eval	400 eval tasks (fit 400 train); unit = task; hash-level	pass@2 80/400 = 20.00%; any-guess 137/400 = 34.25%	secondary, large-N replication
C	Parameterised simulated pools	synthetic	many seeds; known ground truth	oracle by construction	theory stress test + estimator sanity

Table 1. Evaluation datasets (frozen).

3. Theory

3.1 Independent case: top two calibrated probabilities

Theorem 1. If correctness indicators are independent with p_i=P(C_i), then P(¬C_i∧¬C_j)=(1−p_i)(1−p_j), so

U(i,j) = p_i + p_j − p_i p_j, (3)

strictly increasing in each argument. The coverage-optimal pair is therefore the two largest p_i, and attempt order is irrelevant. This is the only regime where first-guess ranking is coverage-optimal — the assumption behind inverse-consensus top-2.

3.2 Duplicates are strictly dominated

If a_2 is the same grid as a_1, C(a_2)=C(a_1) and U(a_1,a_2)=P(C(a_1))=p_{a_1}; any distinct candidate with p>0 strictly increases coverage. **Theorem 2.** Submitting the same grid twice is strictly dominated by any distinct candidate with positive correctness probability.

3.3 Correlation breaks argmax-once

Candidates share TTT state, augmentation view, beam prefix, and model. For two Bernoulli correctness indicators with correlation coefficient ρ,

U(i,j) = p_i + p_j − p_i p_j − ρ√(p_i(1−p_i)p_j(1−p_j)), (4)

from P(¬C_i∧¬C_j)=(1−p_i)(1−p_j)+Cov(C_i,C_j). Two same-view candidates with perfectly correlated failures (ρ=1) force equal marginals p_i=p_j=p: their pair coverage is U=p. An independent third candidate with p_3>0 gives U=p+p_3−p p_3>p; at p=0.50, p_3=0.35 the gain is +17.5 coverage points on that input. **Theorem 3.** Under positive error correlation, the pair maximizing the first guess is not in general coverage-optimal; a decorrelated candidate can strictly improve coverage. For ρ<1 the pairwise preference condition between the runner-up j and a decorrelated candidate k is

(1−p_1)(p_k−p_j) > ρ_{1k}√(p_1(1−p_1)p_k(1−p_k)) − ρ_{1j}√(p_1(1−p_1)p_j(1−p_j)). (5)

Diversification pays when the correlation penalty exceeds the probability advantage — typical for per-view errors; zero or negative correlation does not force it, which is why ρ̂ is measured rather than assumed.

Corollary. If the ranking score is cluster mass and views are independent, top-2 by mass coincides with top-2 by posterior probability, so the rule reduces to inverse consensus — a strict generalization that departs from it under correlation.

3.4 Diversification threshold (Theorem 4)

Theorem 4 (exact, finite-pool). Let two distinct candidates have correctness probabilities $p_1 \geq p_2$ (both in [0,1]) and correctness-indicator correlation $\rho = \mathrm{Corr}(C_1, C_2)$. Then

$$U(\text{pair}) = p_1 + p_2 - p_1 p_2 - \rho\sqrt{p_1(1-p_1)p_2(1-p_2)}, \quad (6)$$

and the pair strictly beats the top-1 **duplicated** attempt ($U_{dup} = p_1$) iff

$$\rho < \rho^*(p_1, p_2) := \sqrt{\frac{p_2(1-p_1)}{p_1(1-p_2)}}. \quad (7)$$

Proof. $U(\text{pair}) - U_{dup} = p_2(1-p_1) - \rho\sqrt{p_1(1-p_1)p_2(1-p_2)}$; its sign is positive iff $\rho^2 p_1(1-p_1)p_2(1-p_2) < p_2^2(1-p_1)^2$, and, for interior p, dividing by the positive $p_1 p_2(1-p_1)(1-p_2)$ gives $\rho^2 < p_2(1-p_1)/(p_1(1-p_2))$; both sides are non-negative for interior p, so taking square roots gives (7). ∎ Equivalently, for interior p_1, p_2, $\Delta U := U(\text{pair}) - U_{dup} = \sqrt{p_1(1-p_1)p_2(1-p_2)} \cdot (\rho^* - \rho)$ — the gain is the distance below the threshold scaled by the candidates' Bernoulli spread. $\rho^* \in [0,1]$ for interior p ($\rho^*=0$ at boundary $p_2=0$ or $p_1=1$, where $\Delta U=0$), with $\rho^* = 1$ iff $p_1 = p_2$; negative correlation always clears the threshold; $\rho = \rho^*$ is exact indifference.

Corollary 1 (equal marginals). If $p_1 = p_2 = p$ then $\rho^* = 1$ and $\Delta U = p(1-p)(1-\rho)$: diversification pays for every $\rho<1$, maximized at $\rho=0$ and worthless at $p \in \{0,1\}$.

Corollary 2 (skew shrinks the threshold). For fixed $\rho>0$, diversification pays iff $p_2 > \rho^2 p_1 / (1-p_1+\rho^2 p_1)$; for small probabilities $\rho^* \approx \sqrt{p_2/p_1}$, so any fixed positive correlation is eventually dominated by skew.

Corollary 3 (score sufficiency and exact-match exclusivity; corrected 2026-10-26). For non-exclusive verifiers, a score tie cannot distinguish candidates, and if correctness is score-determined (Proposition 5) tied candidates share correctness ($\rho=1$, forcing $p_1=p_2$), so $\Delta U=0$. For **exact-match** ARC, distinct candidate grids are mutually exclusive: $C_i \wedge C_j = \varnothing$, hence $\rho<0$ always, the Theorem-4 gate is always met, and $U(a_1,a_2)=p_{a_1}+p_{a_2}$ for any joint distribution; the optimal pair is the two largest calibrated p_i (Corollary 4). The real-pool null is therefore estimation/identifiability, not positive correlation: oracle headroom exists, but $\hat{\rho}$ cannot separate oracle-gap candidates from exposed metadata (Proposition 5).

Derived gate and falsifier. For non-exclusive verifiers, deploy diversification across a relation class only when $\hat{\rho}_c < \rho^*(\hat{p}_1, \hat{p}_2) - \delta$ (δ=0.05, frozen); otherwise take the duplicate/consensus alternative — the gate is derived, not tuned. The empirical instantiation is falsifiable on held-out units: **P1** admitted pairs beat their top-1 duplicate (paired $\Delta>0$, 80% CI>0); **P2** rejected pairs do not ($\Delta \leq 0$); **P3** the predicted-vs-realized gain slope lies in [0.7,1.3]. Failure falsifies the estimator/relation-class model, never the algebra; protocol and tool are in Appendix E. The threshold test ran on dev20 pools, but exact-match mutual exclusivity makes its positive-ρ premise inapplicable: every class has $\hat{\rho} \leq 0$ (same-model/cross-view −0.13, score-tied −0.18), so no deployable pair is rejected and P1/P2/P3 are untestable (§4.8). This is an EXPLORATORY check, not evidence; Table 10 is a prose check and Fig. 9 is omitted.

Table 10 (replaced). Exact-match mutual-exclusivity check on dev20 pools: all relation classes have $\hat{\rho} \leq 0$ (same-model/cross-view −0.13, score-tied −0.18), P2′s rejected set is empty; EXPLORATORY, no P1/P2/P3 claim (theory §4.8).

3.5 Budget-constrained coverage allocation (Theorem 5)

Let $g_t(c)$ be the probability that a pool at per-task compute c covers the truth; expected score is $\Sigma_t\, g_t(c_t)$ under $\Sigma_t\, c_t \leq C$. **Theorem 5.** (i) Identical concave $g_t \Rightarrow$ uniform $c_t = C/T$ is optimal (Jensen). (ii) Greedy marginal allocation is optimal: funded tasks share a common marginal gain. (iii) If margins are ordered by prior accuracy p_t ($p_s \geq p_t \Rightarrow g'_s(c) \geq g'_t(c)$), compute is monotone in p_t and a difficulty-threshold rule holds: only tasks above a budget-dependent cutoff p^*

receive compute, and p* rises as C/T shrinks. (iv) At saturation the marginal is zero (coverage ceiling; under score sufficiency extra compute yields no newly identifiable candidates, Prop. 5), so excess is better reallocated; never-solvable tasks receive none.

3.6 Calibrated plug-in estimator

Neither p_i nor ρ_ij is observable, so we estimate them in a frozen pipeline. (i) $\hat{\mathbf{p}}$: logistic regression [10] on frozen within-input features (score rank, agreement, cluster mass, shape), fit on dev only. (ii) $\hat{\boldsymbol{\rho}}$: per relation class from the dev co-failure statistic, clipped to [−0.95,0.99], with frozen pooling and a default for sparse classes. (iii) $\hat{\mathbf{U}}$: the plug-in version of (4); attempt 1 is argmax p̂, attempt 2 maximizes Û over distinct grids. (iv) **Gate/fallback**: deploy only if dev cross-fit calibration has Brier ≤0.10, reliability slope in the code's valid range [0.2,5.0] (design target [0.7,1.3]) and an estimable relation class; otherwise use a no-calibration fallback (highest agreement, then a different grid). The claim is pre-registered at ≥+1.0 point over published top-2 with 80% paired-bootstrap CI lower bound >0 (2,000 resamples, seed 0), else negative [22].



Figure 4. Illustrative schematic of the coverage rule: candidate pool grouped by relation class, calibrated p̂/ρ̂, pair coverage U, and the chosen distinct pair. Toy values only — not measured.

3.7 TTT as amortized Bayesian adaptation

Why are TTT pools plausible? Read ARC-SFT training as approximate posterior inference [13]: the parameters are a prior over task rules, and per-task TTT performs amortized Bayesian adaptation. A LoRA update is a low-rank posterior correction; a PAC-Bayes sketch predicts an interior optimum in rank/steps and larger gains where the prior is weakest. Test-time augmentation is an equivariance prior, valid only where the task rule commutes with the transform; the frozen κ_T proxy records this. It is a sketch, not a bound.

---

4. Accuracy: the offline TTT system and a real public submission

4.1 First submission attempt (errored)

We entered [15] with one MIT-0 notebook (Qwen3-4B ARC-SFT, D4×S10 augmentation, per-task LoRA TTT, threshold-DFS decoding, NLL rescoring, coverage selection, schema validation). Submission 57456856 (kernel v8; packet `3774c892…`; S1/aug8) used the 41,400 s budget and 578 s/240-task cap (1156 s/120); hidden size unknown. Kaggle reports COMPLETE at ≈02:35 UTC after ≈2 h 34 min, **no public-Score**, errorDescription *"Your notebook hit an unhandled error while rerunning your code…"*. Cause: rerun branch `ARC_BUDGET_S="41400.0"` parsed by `int("41400.0")` ⟶ `ValueError` before try/except. The hardened packet adds int-typed/defensive parses, no-raise validation and a SAFETY cell; probe 57500870 scored 0.00. **We claim no score for 57456856 and report the failure as engineering, not model quality.**

Final hidden-test results (six scored submissions): **S5 16.25** (ref **57681632**, kernel v16; champion), **S6 15.42** (57716141), **S5_ens 15.42** (57752011), **S1 14.17** (57663405), **S5+LegC 14.17** (57785740), CPU probe **0.00** (57500870); plus one invalid-format attempt (57442788) and one rerun error (57456856, Table 9). No score exceeds 16.25. A reference public-notebook replication attempt reached **3/20** on dev20, below our S1 4/20, so the ~30 public-notebook band is external context, not a matched reproduction. The LegC verified-induction hybrid was **net-negative** on the hidden half (−2.08 vs S5) and was rejected. Compute quota was exhausted Oct 31–Nov 1, leaving no GPU for tuning; the Nov 2 final entry uses the best completed version (S5 v16) unless S5b beats 16.25. Half-cost configs truncate at the 578 s cap: these scores are far below the ~30 band, not SOTA.

57456856 exposes no per-task diagnostic; labeled diagnostics come from dev pools (§4.3) and third-party pools (§5).

date (UTC)	ref	config	LB	status / notes
2026-10-20T14:56	57442788	CPU-fallback insurance	—	invalid format
2026-10-21T00:01:15	57456856	S1 v8, packet `3774c892…`	—	rerun-only `int("41400.0")` ValueError; no public-Score
2026-10-22T00:00:23	57500870	S1 CPU-version probe	**0.00**	COMPLETE, no error
2026-10-26T10:45	57663405	S1 v8 L4, packet `9060264c…`	**14.17**	COMPLETE, no error (≈14 h incl. queue)
2026-10-27T00:00:28	57681632	**S5 v16 L4**, packet `6211b694…`	**16.25**	COMPLETE, no error — **champion**
2026-10-28T00:00:27	57716141	S6 dual-config v22	**15.42**	COMPLETE
2026-10-29T00:02:23	57752011	S5_ens two-model v23	**15.42**	COMPLETE
2026-10-30T00:02:21	57785740	S5+LegC hybrid v25	**14.17**	COMPLETE — LegC net-negative
2026-10-31	—	(submission missed)	—	session gap; no ref
2026-11-01T00:10:23	57854697	S5b v24, packet pending	pending	PENDING at freeze — last S5-family probe

Table 9. Submission ledger (all public-LB attempts; errors and misses included).

Hidden-set size remains unknown; runtimes are queue time, not a size signal. Six scored entries; all scores are public-LB hidden-half results. Table 8′s dev20 scores are public-eval-subset measurements, not hidden scores.

4.2 Hidden-test budget calibration

Hidden size is unknown (120–240); for the 240/259 sample shape at 11.5 h, the four-worker budget is 41,400 s and the fair per-task cap is 578 s at 240 tasks (1156 s if 120; 9.6/19.3 min), including a reserve. The cap truncates attempts. Dev2 timings: ≈220 s/task (202.51/238.12 s). Scaling to 240 tasks on four workers gives ≈3.7 h of TTT — 2.6× per-task headroom, ~7.8 h unused wall. Slack absorbs harder tasks, model load (106.9 s) and worker skew. Processed-task count and truncation are not observable from the rerun interface. Figure 8 reserves this calibration and the §4.3 dev compute-accuracy points. Theorem 5 predicts compute beyond a coverage plateau is wasted; the dev20 ladder tests this (Table 11; EXPLORATORY/PENDING).

4.3 Compute-normalized dev evidence

On this single 20-task subset (no CI claim), the all-config union (incl. S5_ens) reaches 6/20 (30%) versus 4/20 for S1; four-config S1/S2/S5/S6 union 5/20. Dev2 pilot: 466.7 s wall, 106.9 s load, 15 candidates for 2/2 inputs, oracle 1/2 — plumbing evidence, not accuracy.

config	dev20 (task)	dev20 (input)	final LB	notes
S1 (hardened)	4/20 = 20.0%	6/32 = 18.8%	**14.17**	decode mean ≈235 s/task; 4 tasks capped
S2 (aug16)	2/20 = 10.0%	5/32 = 15.6%	—	decode up to 618 s; truncation
S5 (single)	3/20 = 15.0%	5/32 = 15.6%	**16.25**	champion
S6 (v13; selection-bug)	2/20 = 10.0%	5/32 = 15.6%	**15.42**	pass@2 flagged; 4/20 oracle, usable
all-config union	6/20 = 30.0%	—	—	cand.-union incl. S5_ens; four-config 5/20

Table 8. dev20 sweep (eval20: 20 tasks / 32 inputs; L4×4; hardened packets).

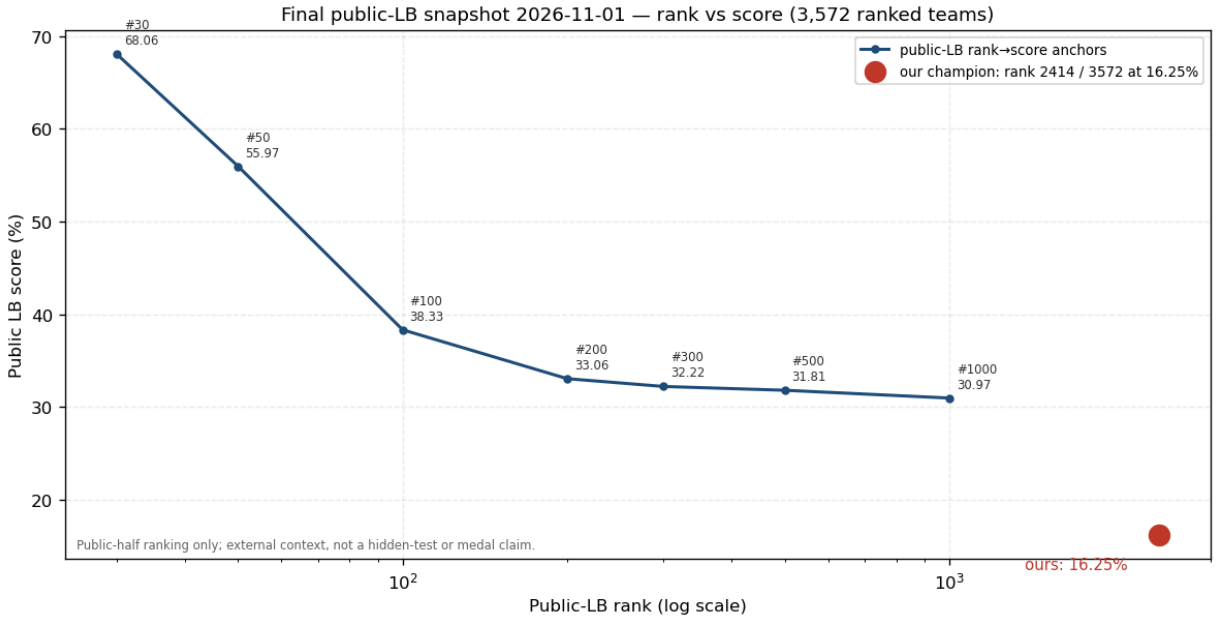


Figure 8. Final public-LB snapshot (2026-11-01T00:13Z): rank↔score anchors and our champion at rank 2414 / 3,572 (score 16.25). Public-half ranking only; external context, not a hidden-test or medal claim.

Table 11. Public-LB snapshot 2026-11-01T00:13:02Z. 3,572 ranked teams; leader 88.06. Our row: `rank 2414 · "wen Chen" · TeamId 17156552 · score 16.25 · submissions 8 · last 2026-10-30 00:02:21` (three teams tied at 16.25, ranks 2412–2414; 67.5th percentile from top). Anchors: r30 68.06, r50 55.97, r100 38.33, r200 33.06, r500 31.81, r1000 30.97. The public LB displays our **best** score (16.25), not the last submission (v25 = 14.17): standings use best-of-submissions semantics, so the Nov 2 entry is only a safety copy for the last-submission rule. The ~30–31 notebook band is ≈ranks 500–1000 (top 14–28%); 16.25 is far below it — public-half ranking only; no medal/SOTA claim.

5. Empirical study: selection rules on third-party candidate pools

5.1 Sources, reproduction gates, and protocol

Released third-party logs bound the study: gains are pool-conditioned; sources are frozen and a failing gate excludes one. Table 3 gates: B's coverage summary (29 inputs; oracle 9/29; top-2 8/29) and A's artifact (137/400 solved; 80/400 pass@2) reproduce exactly. Rules: duplicate, argmax, top2, cal_indep, coverage_optimal, oracle_any2 (ceiling), deployed. Splits are task-level (B 9/11 tasks, 13/16 inputs; A 400/400); paired bootstrap over tasks (2,000 resamples, seed 0) [14] gives 80%/95% CIs; no refits after holdout.

source	gate	published	our reproduction	pass
B	coverage summary	29 inputs; oracle 9/29; top-2 8/29	exact, 0 per-input mismatches	yes
A	upstream script, 2000 steps	solved 137/400; ≤2 guesses 80/400	exact via bundled artifact	yes

Table 3. Reproduction gates (source-published metric vs our re-derivation).

5.2 Pilot evidence on Dataset B

B is the only public candidate-dump notebook; 20 tasks, pilot-only, holdout 16 inputs/11 tasks. Oracle 31.25% (5/16); every deployable rule (duplicate, argmax, top-2, cal_indep, coverage) also 31.25%, indistinguishable (80% CI [18.75,43.75]); headroom 0. Deployed fallback 25.00%.

5.3 The large replication and the simulated stress test

A is claim-bearing: top-2 covers 80/400 (20.00%) vs 137/400 any-guess (34.25%). The result is a null: correctness is predictable but score-dominated. Missed truths are far from the top (rank median 12). `DOWNGRADE_TO_ANALYSIS`; deployed fallback 19.25%; CompressARC [6] is ARC-AGI-1-era (hash-level pools).

C isolates the estimator; a sanity check, not evidence: calibrated rules reach the oracle 41.67% (+18.33 pp, 95% CI [11.67, 25.0]; gate slope 0.025).

rule	B: ARC-AGI-2 TTT pools (16 inputs / 11 tasks)	A: CompressARC (400 tasks)	C: simulated (120 tasks)
oracle_any2 (ceiling, not a rule)	31.25 [18.75, 43.75]	34.25 [31.25, 37.50]	41.67 [35.83, 47.50]
duplicate	31.25 [18.75, 43.75]	18.50 [16.00, 21.00]	16.67 [12.50, 20.83]
argmax1_argmax2	31.25 [18.75, 43.75]	19.50 [17.00, 22.00]	23.33 [18.33, 28.33]
top2_consensus	31.25 [18.75, 43.75]	20.00 [17.50, 22.75]	21.67 [16.67, 26.67]
cal_indep	31.25 [18.75, 43.75]	18.00 [15.50, 20.75]	41.67 [35.83, 47.50]
coverage_optimal	31.25 [18.75, 43.75]	17.75 [15.25, 20.25]	41.67 [35.83, 47.50]
deployed (gate-selected)	25.00 [12.50, 37.50]	19.25 [16.75, 21.75]	41.67 [35.83, 47.50]

Table 2. Frozen two-attempt rules on the three datasets (pass@2, %; 80% bootstrap CI in brackets; seed 0, 2,000 resamples; two decimals).

Cells are pass@2 from the frozen artifact `analysis/results/selection_benchmark.json` (sha256 `6c16abfef7f0efe3a1a6bbe9f6251c9aeef1893ee7b0f44fdfdb9c2eb34ab71a`; 80% CIs). C is simulated, not evidence about real solvers.

Score sufficiency explains the null: score ordering saturates recoverable coverage, so selection is not the bottleneck — generation is; re-ranking never exceeds top-2′s 20.00%, the 14.25-point headroom is real but not separable from exposed metadata. Falsifiers: (F1) a calibrated rule beating the solver by ≥+1.0 point (80% CI lower >0); (F2) metadata exposing oracle-gap candidates; (F3) a multi-decoder union exceeding noise.

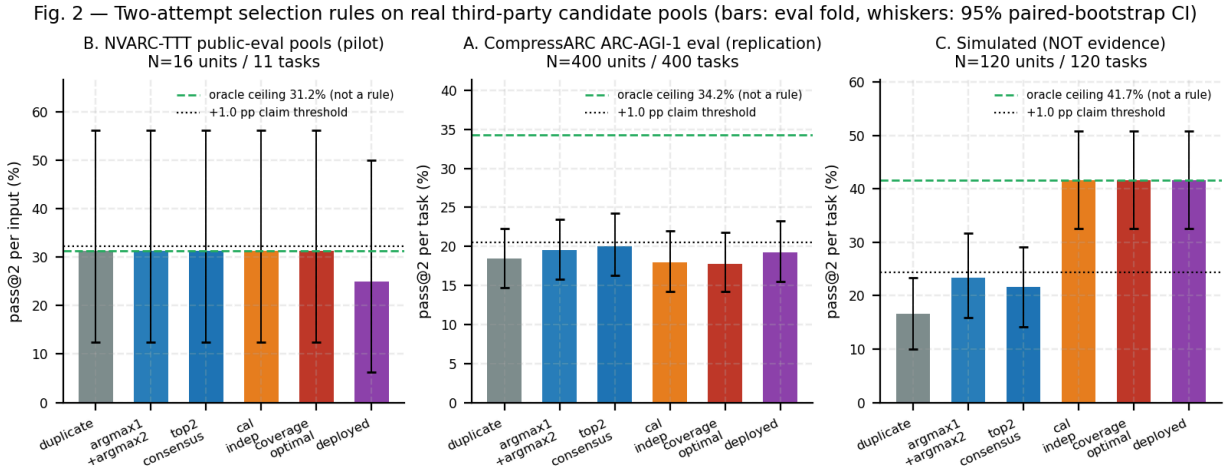


Figure 2. Rule comparison with bootstrapped CIs: pass@2 and paired Δ vs the published baseline, per dataset (B pilot; A replication; C simulated). Oracle coverage is a ceiling, not a rule; bars are the pre-registered 80%/95% intervals; the +1.0-point line applies to the claim-bearing dataset only.

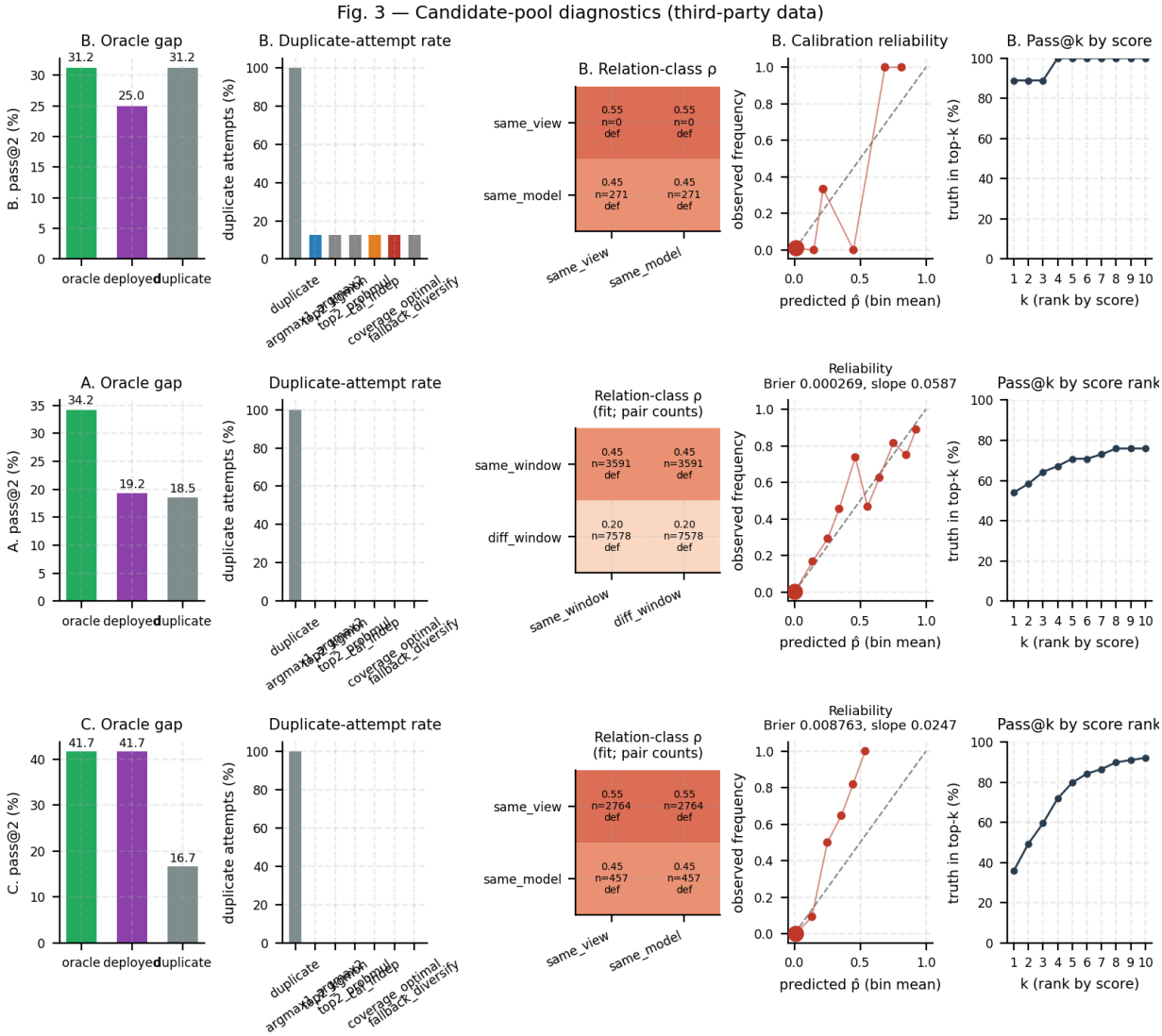


Figure 3. Pool diagnostics: oracle gap, duplicate rates, relation-class ρ heatmap with pair counts, and calibration reliability for B/A and C.

6. Data characterization and leakage audit

We characterize ARC-AGI-2 data [20]: 1,000 training tasks (3,232 pairs, 1,076 inputs) and 120 evaluation tasks (358 pairs, 172 inputs); the repo mirror (167 inputs) is not used for scores. A SHA-256 audit finds zero shared tasks, pairs and input grids between training and evaluation.

Heuristic baselines solve 1.6% training, 1.5% holdout and 0% eval; eval grids are larger (21.95 vs 14.67, p<0.001) and 54.5%/57.5% of train/eval tasks are same-size rewrites.

quantity	value	artifact
train / eval tasks	1,000 / 120	§1
shared tasks / pairs / input grids	0 / 0 / 0	§12
trivial-heuristic solve: train / holdout / eval	1.6% / 1.5% / 0%	§13
max grid dimension: train vs eval	14.67 vs 21.95 (p<0.001)	§11
solvability tiers A/B/C	28 / 56 / 36 (uncalibrated)	§15

Table 6. Data-characterization summary (public ARC-AGI-2; unless noted).

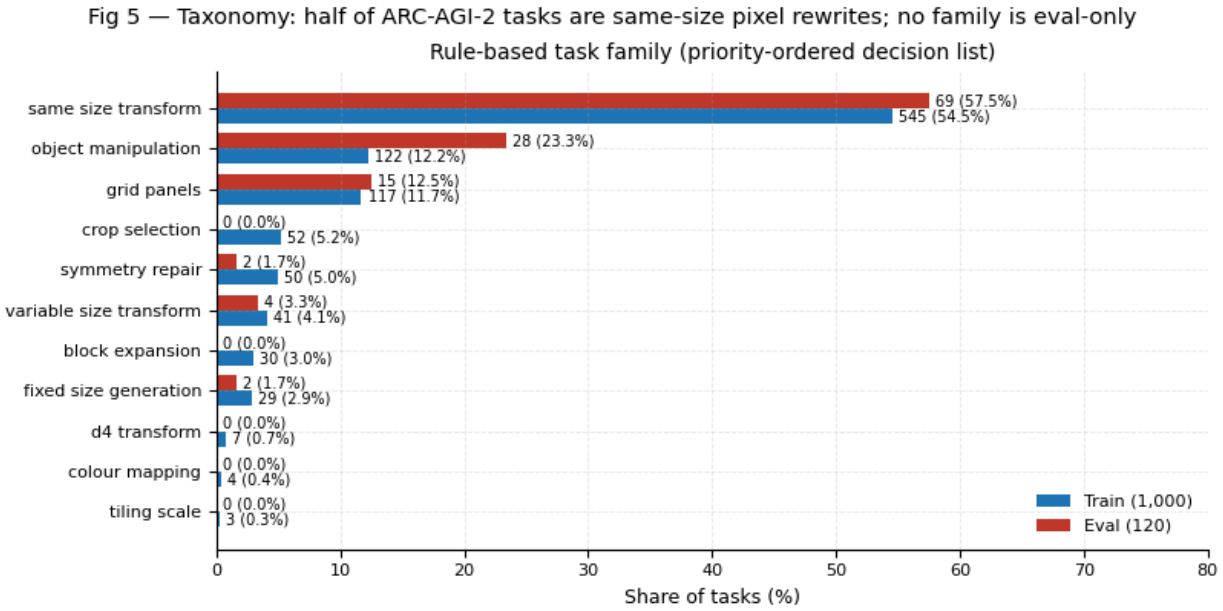


Figure 5a. Rule-based task-family distribution for 1,000 training and 120 public-evaluation tasks (analysis/report.md §9). Same-size rewriting dominates both; no family is evaluation-only.

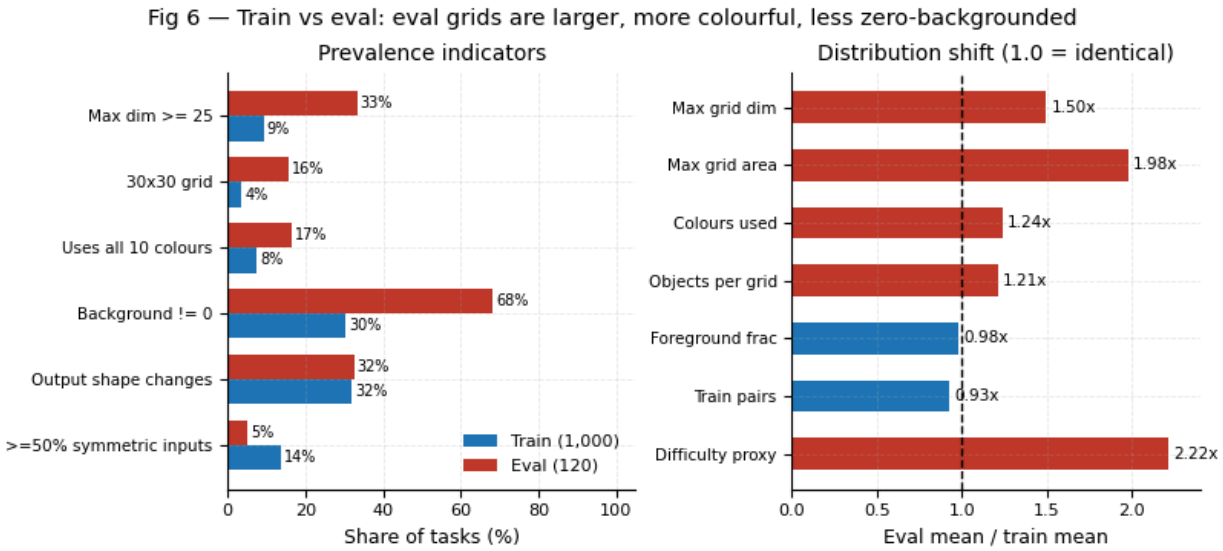


Figure 5b. Train-vs-evaluation shift: prevalence indicators (left) and eval/train mean ratios (right) for size, palette, demonstrations, and background (analysis/report.md §11).

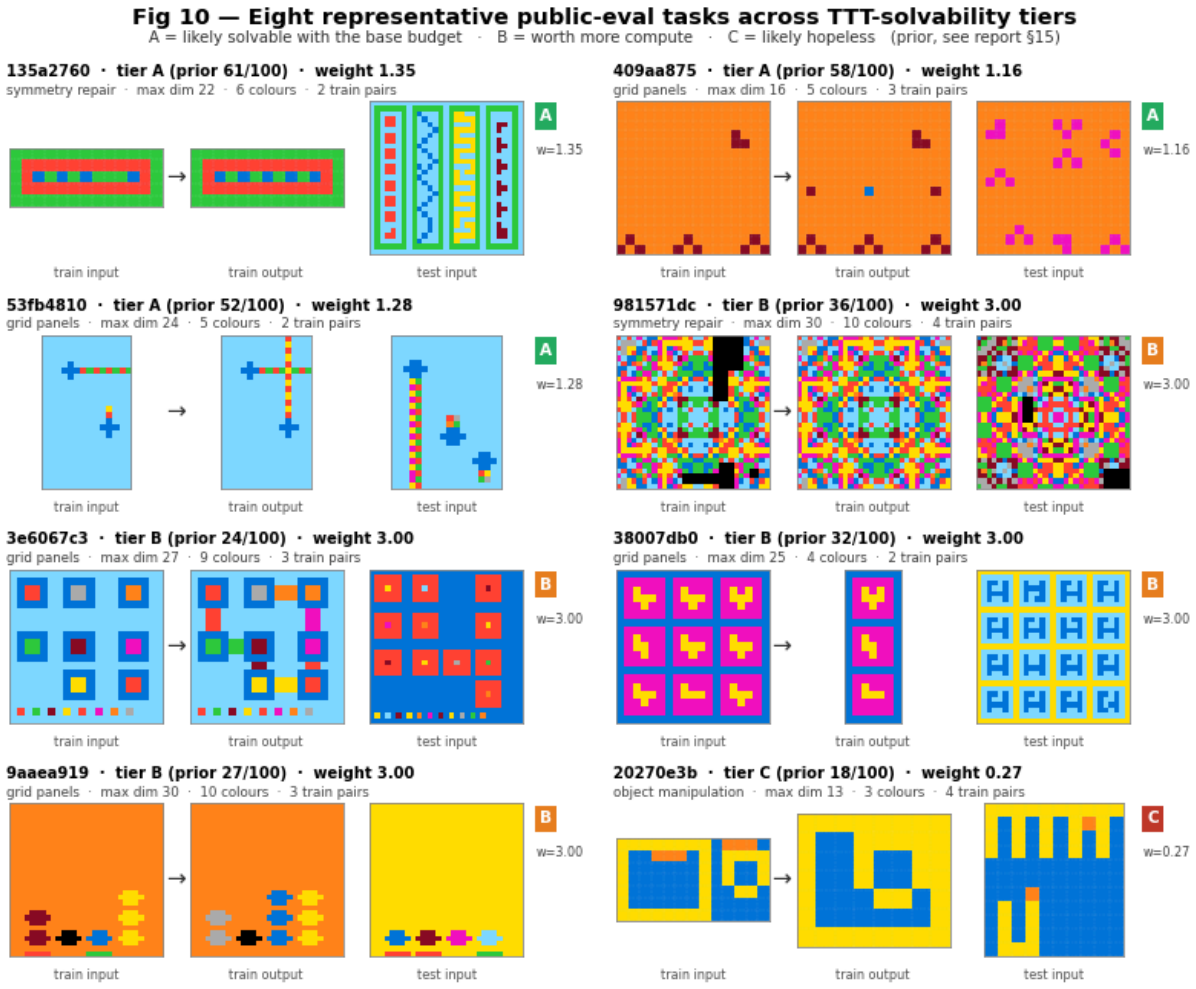


Figure 6a. Eight representative public-evaluation tasks across prior tiers with budget weights (analysis/report.md §15); tiers are an uncalibrated prior, not a solve rate.

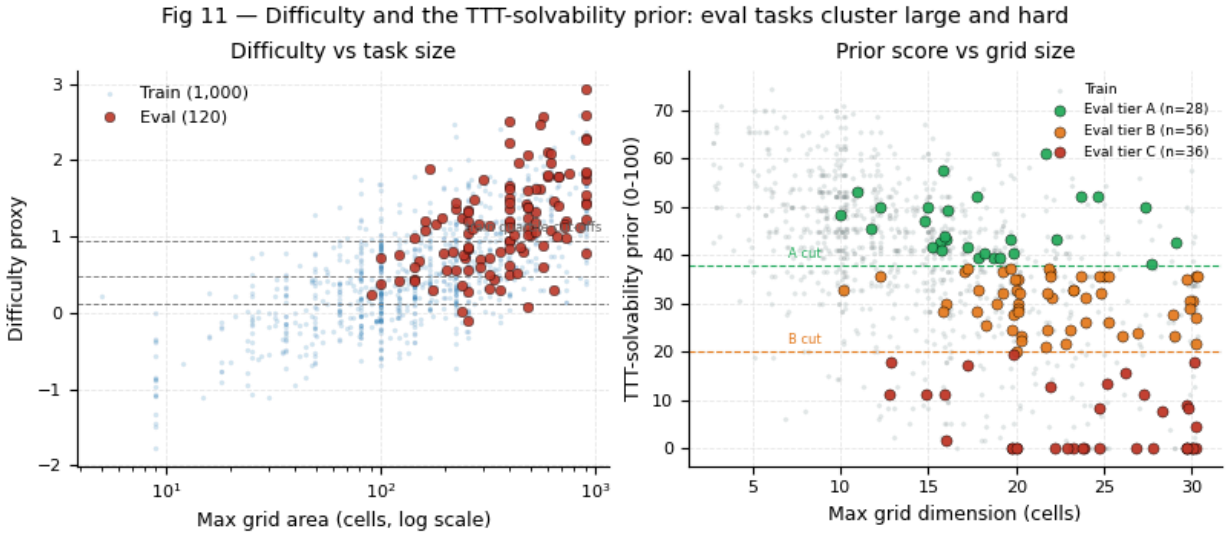


Figure 6b. Difficulty proxy and the uncalibrated TTT-solvability prior versus task size (analysis/report.md §15–16).

7. Completeness: engineering and reproducibility

The repository implements loader, augmentation, threshold-DFS decoding, rescoring, aggregation, coverage rules (§3), TTT and orchestration; it runs offline from a kernel mount. Released under

MIT-0; third-party components retain licences (ARC-AGI-2, Qwen3, CompressARC, arc-dsl, Unsloth/peft). Notebook methods are clean-room re-implemented and cited; a NOTICE records attributions. The `LICENSE` file must switch to SPDX `MIT-0` before release.

GPU fixes: Triton needs an explicit `TRITON_PTXAS_PATH=/usr/local/cuda/bin/ptxas` (missing ptxas → zero candidates), and Hugging Face loading needs offline weights (`local_files_only`/mount). The packet pins source/hash and forces deterministic fallback; Validation: 190 passing CPU tests (~26 s); a schema-valid fallback: the 167-input packet ran in 8.3 s on Kaggle, while the official 172-input repackaging measured 5.4 s locally and scores 0/120 — a safety net, not accuracy; TTT-path validation with a tiny Qwen model (dialect guard, LoRA, 2 steps, 8-view decode, selection, schema); and a CompressARC harness that re-derives bundled predictions and exports hash-level pools for Dataset A.

component	validation evidence	status
CPU test suite	190 tests pass (~26 s)	measured
CPU fallback submission	repo 167-input packet 8.3 s on Kaggle; official 172-input repackaging 5.4 s local, 0/120 (safety net)	measured
TTT path (tiny model)	tokenizer guard, LoRA, decode, selection, schema OK	measured
GPU dev2 pilot	2 tasks, 15 candidates, per-task walls 202.5/238.1 s, load 106.9 s	measured
Engineering fixes	ptxas/HF-offline guards; void stale-packet run root-caused	measured
First submission attempt	57456856 errored (rerun-only budget parse); CPU-probe retry (57500870) publicScore 0.00	measured failure
First scored hidden run	**57663405**, S1 L4, COMPLETE **publicScore 14.17** (≈14 h incl. queue)	measured (scored)
Best scored hidden run	**57681632**, S5 L4, COMPLETE **publicScore 16.25** (current best; +2.08 pp over S1)	measured (scored)
S6 dual-config rerun	Oct 28 submission	PENDING
CompressARC reproduction	137/400 solved; 80/400 pass@2	measured

Table 5. System and reproducibility matrix.

8. Public-solver landscape and what transfers

Table 4 has ten notebooks [16–19]; nine cluster at 29.7–31.4. TRM [5], SOAR [7], deep ensembles [11] are not evaluated.

Saved-artifact official-label scores are partial reruns (vanductan 5/120; 8/172; others 2–4/120).

notebook (family)	displayed	claimed	mechanism	transferable element
mayankraaz	31.39	35.97	triple-pass pooled TTT, controller, LOO-DSL	cheap-first ordering; pass-A invariant
yuvrajlamba01	31.39	54.31%	TTT + deterministic symbolic solver	monotone symbolic filler
rabari9999	30.97	n/a	v38 perf patch clone	GPU-side log-softmax patch
vanductan	30.97	n/a	perf-patched research dump	candidate-pool logging (Dataset B)
mikelou1 / anthonyxlyu	30.14	33.89	v38 minimal perf patch	algebraically identical speed patch
woominyo	29.72	n/a	perf patch + interleaved ensemble	attempt ordering via interleaving
dmitriigluzdov	29.17	n/a	+6 transformer layers	negative result — capacity expansion
delongyang	15.69	n/a	27B FP8 zero-shot	model-size/throughput trade-off
cprs	30.56	80%+ target	TTT + LLM program-synthesis pre-pass	verified-program candidate source

Table 4. Public-solver landscape (dated reading 2026-10-18; displayed scores are secondary-source context).

9. Limitations, ethics, and reproducibility

The largest limitation is that our solver has **six scored hidden submissions** spanning **14.17–16.25** (champion S5 16.25), versus **20.0% task-level** on the 20-task public-eval subset — S1/S5 gaps ≈5.8/≈3.75 points consistent with half-cost truncation and subset noise, not distributional. Compute quota was exhausted Oct 31–Nov 1, leaving no GPU for further tuning; the ~30 public-notebook band is external context only. The first rerun (57456856) errored after ≈2 h 34 min with no publicScore; probe 57500870 scored 0.00. No per-task hidden observability exists, so variance cannot be reported. The selection claim remains a third-party-pool study with narrow conclusions; B is tiny (29 inputs/20 tasks), A overlaps ARC-AGI-2 training, C validates only estimator/implementation.

The theory is bounded: coverage identities are exact, but $\hat{p}$ and $\hat{\rho}$ are estimates; the gate is a guard, not a guarantee; the TTT discussion is a sketch.

All data are public (ARC Prize data, notebook outputs, CompressARC artifacts); no personal data, credentials or hidden-test information is used. Third-party artifacts are used with attribution; no third-party code was copied. Reproducibility depends on Appendix B artifacts and on the repository reaching a single consistent licence before release.

10. Conclusion

Coverage is the right formalization for two-attempt selection: independent correctness makes top-2 optimal, correlated errors make a calibrated pair rule provably better, and Theorem 4 gives the correlation threshold. The measured answer is a bounded negative: on real public pools the solver's score ordering saturates recoverable coverage. On 400 CompressARC tasks top-2 covers 20.00% while the coverage rule covers 17.75%, against 34.25% oracle — 14.25 points exposed metadata cannot reach. The verdict is `DOWNGRADE_TO_ANALYSIS`; selection is not the bottleneck, candidate generation/coverage is. Transferables: the formalization, protocol, benchmark, data audit, implementation, real attempt and record — plus a falsification target: a pool with independent evidence, or features exposing oracle candidates the score hides.

References

1. Chollet, F. *On the Measure of Intelligence.* arXiv:1911.01547, 2019. https://arxiv.org/abs/1911.01547

2. ARC Prize. *ARC Prize 2026 competition and paper-track rules.* https://arcprize.org/competitions/2026 (accessed 2026-10-18).
3. Sorokin, I., Puget, J.-F. *NVARC solution to ARC-AGI-2 2025* (Kaggle 1st place; model `sorokin/qwen3_4b_grids15_sft139`, Apache-2.0), 2025. https://www.kaggle.com/models/sorokin/qwen3_4b_grids15_sft139 ; https://github.com/1ytic/NVARC
4. Koushik Rudra. *failed-in-aimo* public Kaggle notebook (NVARC-style TTT reference), 2026. https://www.kaggle.com/code/koushikrudra/failed-in-aimo
5. Jolicoeur-Martineau, A. *Less is More: Recursive Reasoning with Tiny Networks (TRM).* arXiv:2510.04871, 2025. https://arxiv.org/abs/2510.04871
6. Liao, I., Gu, A. *ARC-AGI Without Pretraining (CompressARC).* 2025. https://iliao2345.github.io/blog_posts/arc_agi_without_pretraining/ ; code (MIT): https://github.com/iliao2345/CompressARC
7. Pourcel, J., Colas, C., Oudeyer, P.-Y. *Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI (SOAR).* arXiv:2507.14172, 2025. https://arxiv.org/abs/2507.14172
8. the ARChitects. *ARC-AGI-2 2025 solution writeup.* 2025. https://lambdalabsml.github.io/ARC2025_Solution_by_the_ARChitects/
9. Cole, J., Osman, M. *Don't throw the baby out with the bathwater: How and why deep learning for ARC.* arXiv:2506.14276, 2025. https://arxiv.org/abs/2506.14276
10. Guo, C., Pleiss, G., Sun, Y., Weinberger, K. *On Calibration of Modern Neural Networks.* ICML 2017. https://arxiv.org/abs/1706.04599
11. Lakshminarayanan, B., Pritzel, A., Blundell, C. *Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles.* NeurIPS 2017. https://arxiv.org/abs/1612.01474
12. Snell, C., et al. *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters.* arXiv:2408.03314, 2024. https://arxiv.org/abs/2408.03314
13. Akyürek, E., et al. *The Surprising Effectiveness of Test-Time Training for Abstract Reasoning.* arXiv:2411.07279, 2024. https://arxiv.org/abs/2411.07279
14. Efron, B., Tibshirani, R. *An Introduction to the Bootstrap.* Chapman & Hall, 1993. DOI: 10.1201/9780429246593
15. Kaggle. *ARC Prize 2026 ARC-AGI-2 code-competition metadata* (kernels-only, no-internet rerun). https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2 (accessed 2026-10-18).
16. vanductan. *arc-agi-2-candidate-dump* public Kaggle notebook (displayed LB 30.97; candidate-pool research dump), 2026. https://www.kaggle.com/code/vanductan/arc-agi-2-candidate-dump
17. mayankraaz. *arc-agi-2-triple-pass-nvarc-sota-35-9-lb* public Kaggle notebook, 2026. https://www.kaggle.com/code/mayankraaz/arc-agi-2-triple-pass-nvarc-sota-35-9-lb
18. woominyo. *arc26-nvarc-lb33-4xl4* public Kaggle notebook, 2026. https://www.kaggle.com/code/woominyo/arc26-nvarc-lb33-4xl4
19. mikelou1. *arc-agi2-lb33-89-minimal-perfpatch* public Kaggle notebook, 2026. https://www.kaggle.com/code/mikelou1/arc-agi2-lb33-89-minimal-perfpatch
20. ARC Prize. *ARC-AGI-2 public data release* (Apache-2.0). https://github.com/arcprize/ARC-AGI-2 (accessed 2026-10-18).
21. ARC Prize. *ARC-AGI-2 2026 scoring rules and leaderboard snapshot* (secondary rendering, 2026-10-18). https://arcprize.org/competitions/2026/arc-agi-2
22. ARC-2026 team. *Selection protocol v2: pre-registered evaluation on third-party candidate pools* (internal pre-registration, frozen 2026-10-18; released with the paper's artifact bundle).
23. D. Zhong. *arc-agi2-nvarc-plus-ckpt* (v6s) public Kaggle notebook, 2026 (uses "coverage-aware set selection" and a P4 structural-distinctness attempt-2 rule; the referenced theory notes v32/v33 are not public). https://www.kaggle.com/code/dalezhong/arc-agi2-nvarc-plus-ckpt

24. Wei et al. *FlexRouter: Coverage-Oriented Correlated Subset Selection* (COLM 2026). https://arxiv.org/abs/2609.38585
25. Singham. *Selection Under an Imperfect Verifier: An Extreme-Value Theory of Test-Time Compute.* Zenodo DOI 10.5281/zenodo.21976219, 2026. https://doi.org/10.5281/zenodo.21976219
26. M. N. Jacob. *arc-budget: where test-time compute goes in an ARC-AGI-2 system* (software; empirical instrumentation study, not a formal coverage model), 2026. MIT-0. https://github.com/manunicholasjacob/arc-budget — associated writeup "Most of the Budget Buys Nothing, and a Fifth of What It Finds Is Thrown Away".
27. K. Moghe and P. Chin. *Cost-Effective Agent Harnesses for Abstract Reasoning and Generalization on ARC-AGI-1.* arXiv:2607.06764 [cs.AI], 2026 (preprint; no venue). https://arxiv.org/abs/2607.06764
28. Bay and Yearick. *When More Sampling Hurts: The Modal Ceiling and Correlation Ceiling of Test-Time Scaling.* arXiv:2606.28661, 2026 (coverage-vs-selection ceilings, "identifiability gap"). https://arxiv.org/abs/2606.28661
29. Nemhauser, G.L., Wolsey, L.A., Fisher, M.L. *An analysis of approximations for maximizing submodular set functions—I.* Mathematical Programming 14(1):265–294, 1978.
30. Markbarney et al. *Modality-Driven Search with Holistic Trace Judging for ARC-AGI-2.* 2026. https://arc.markbarney.net/paper.pdf

All web sources accessed 2026-10-18 (refs 26–30: 2026-10-20); arXiv identifiers resolve at `arxiv.org/abs/<id>`.

Appendix A — Pre-registration (condensed from `research/design/selection_protocol_v2.md`)

- **Sources frozen before evaluation:** B = `vanductan/arc-agi-2-candidate-dump` (pilot; 29 inputs/20 tasks — no other public candidate-dump notebook was found); A = CompressARC bundled predictions (claim-bearing; 400 ARC-AGI-1 eval tasks, fit on 400 train; hash-level candidates).
- **Reproduction gates:** B must reproduce 29 units, oracle 9, top-2 8; A must reproduce 137 solved and 80 pass@2 at 2,000 steps. A failed gate excludes the source; it does not trigger rule changes.
- **Units and splits:** B unit = test input, tasks split 9/11 dev/holdout by task hash (13/16 inputs); A unit = task, source-defined split (400/400).
- **Frozen scalar scores:** B = Σ of the eight per-augmentation scores (lower better); A = logaddexp-accumulated pick score (higher better).
- **Rules:** duplicate, argmax1_argmax2, top2_consensus, cal_indep, coverage_optimal, oracle_any2, deployed (selected on fit).
- **Features for $\hat{p}$ (fixed order):** score rank percentile, score z-score, log(1+agreement), cluster mass, is-top-cluster, shape consistency (B), cross-model count; A adds log first-step and pick-type diversity.
- **ρ classes:** same_candidate (ρ=1, never paired), same_view, same_source, cross_source (pooled only; no cross-source claim).
- **Statistics:** paired bootstrap over tasks, 2,000 resamples, seed 0; 80% and 95% CIs; claim iff $\Delta \geq +1.0$ point and 80% CI lower > 0 on the claim-bearing holdout.
- **Verdict:** Dataset A does not meet the pre-registered margin — coverage_optimal 17.75% vs argmax1_argmax2 19.50%, margin −1.75 pp (required: ≥+1.0 point with 80% CI lower bound >0) — so the selection claim is `DOWNGRADE_TO_ANALYSIS` and the deployed rule is the no-calibration fallback. Dataset C is simulated and therefore never enters the claim.
- **Deviations:** any change is a new protocol version committed before the affected holdout is read; both versions reported.

Appendix B — Evidence registry and per-number provenance

All numbers map to rows in `paper/evidence_table.md` (Plan C). Primary artifacts: `analysis/results/selection_benchmark.json` (Table 2 and Fig. 2; command `python analysis/selection_benchmark.py`); `logs/arch-theorist/external_reproduction_gate_arc2_dump.json` (Dataset B reproduction gate); `code/out/compressarc/verify_bundled_eval.json` (Dataset A reproduction gate); `analysis/report.md` (data characterization); `analysis/results/saved_artifact_scores.json` (landscape); `logs/kaggle_freeze.md` and `logs/kaggle_runs.md` (platform gate); `code/docs/RESULTS.md` (CPU baselines and system validation). The theorem and the pre-registration are documented in `research/design/theory.md` and `research/design/selection_protocol_v2.md`, released with the paper bundle. GPU end-to-end execution and our own leaderboard score are **not measured** (platform-gated) and are never cited as results.

Appendix C — Platform-gated access log (summary)

Thirteen Kaggle runs recorded between 14:00 and 15:06 UTC on 2026-10-18, all CPU-only: five environment/mount/tokenizer probes, three fallback-packet dry runs, five TTT-packet smoke runs. GPU entitlement was requested (`NvidiaL4`, `NvidiaTeslaT4`) and not granted; competition attach was rejected pending rules acceptance, which requires phone verification. Zero Kaggle GPU-hours were consumed; the frozen handoff bundle contains packet checksums and a resume procedure that regenerates packets before pushing. No hidden-test data was accessed and no GPU number appears in the paper. **V4 update (2026-10-21):** the competition gate cleared; kernel v8 dev2 pilot measured on 4×L4 (§4.3). Submission **57456856** reran (2026-10-21T00:01:15Z) and errored before compute: the rerun branch set `ARC_BUDGET_S="41400.0"` and the run cell's `int("41400.0")` raised `ValueError`; Kaggle's ≈2 h 34 min was queue time. Hardening (int-typed budgets, no-raise validation, sanitization, SAFETY cell, rerun-simulation gate) landed; probe 57500870 scored 0.00; L4 refs 57663405=14.17, 57681632=16.25; S6 15.42.

Appendix D — Reproduction commands

```
# data analysis, split, baselines, allocation, figures (CPU, 169 s)
bash analysis/run_all.sh

# selection benchmark on third-party pools (CPU; needs the downloaded dumps)
python analysis/selection_benchmark.py

# repository tests and CPU baselines
python -m pytest code/tests -q
python code/compressarc_harness.py verify-bundled --out code/out/compressarc/
verify_bundled_eval.json
```

Appendix E — Diversification-threshold derivation, P1/P2 protocol, and falsifier

Theorem 4 and its corollaries are stated in §3.4; the full derivation, edge cases, and the exact inversion of the threshold are in `research/design/theory.md` §4.2. The empirical protocol is frozen in `research/design/selection_protocol_v2.md` §6d and implemented in `research/design/diversification_threshold.py` (self-test covers theorem arithmetic, admission/bootstrap mechanics and pilot flagging). On `pool-v1` logs from our own GPU runs: unit = (task, test input); task-level fit/holdout split by `sha256(task_id)` parity; $\hat{p}$ from the frozen dev-fit logistic calibration and $\hat{\rho}_c$ from the method-of-moments co-failure estimator per relation class (same_view, same_model_cross_view, score_tied, cross_model), clipped to [−0.95,0.99]; δ=0.05; pairs cap at the top 32 by $\hat{p}$. Rules compared on holdout units: top-1 duplicated, threshold-diversify (admit iff $\hat{\rho}_c < \rho^* - \delta$), and the ungated coverage-optimal contrast. Pre-registered predictions: **P1** admitted pairs beat their duplicate counterpart (paired $\Delta>0$, 80% CI>0); **P2** rejected (score-tied) pairs do not ($\Delta\leq 0$); **P3** predicted-vs-realized gain slope in

[0.7,1.3]. Falsifier: failure of P1, P2 or P3 falsifies the estimator/relation-class instantiation, never the algebra. The dev20 pools (65 base tasks / 133 task-pools) were analyzed; exact-match exclusivity makes P1/P2/P3 untestable (theory §4.8): every class has $\hat{\rho}<0$, so Table 10 is a prose check and Fig. 9 is omitted.

Appendix F — Official competition data note

The official evaluation files (`data/comp/`, sha256 prefixes `e7c62a4b…` / `84be4f4f…`) contain 120 tasks and **172** test inputs; the repo mirror distributed with the public data has 167. Five tasks differ: four add an extra test input (inserted first, so attempt order shifts), `faa9f03d` also differs in its train pairs, and `d8e07eb2` has a different train-pair output. Repo-mirror scores are therefore not official scores, and all official numbers in this paper are computed by `analysis/official_scoring.py --recompute-all`. The local `arc-agi_test_challenges.json` file (240 tasks / 259 inputs) is an exact content duplicate of 240 public training tasks and is a local dev sandbox only; it is never compared to the hidden leaderboard. Rerun budget: 4 workers × 12 h for ~240 tasks / 259 inputs gives 12.0 min per task / 11.1 min per input (10.8 / 10.0 min with a 10% margin). Dataset B's selection benchmark attached truth from the repo mirror; the one differing task present in B (`abc82100`) has no candidate matching any repository or official truth for its decoded input, so no rule covers it and the B row is unchanged (checked in `paper/official_b_truth_check.md`).

Appendix G — Theorem 5 derivation, dev20 protocol, and falsifiers

Appendix G / Table 11 (reserved): Theorem-5 derivation (Jensen, KKT/exchange, single-crossing, saturation); dev20 ladder S1/S2/S5/S5_ens ⟶ per-task wall, candidates, coverage, marginal gains by $\hat{p}$ proxies; PENDING/EXPLORATORY.

Appendix H — Exploratory cross-source note

An exploratory check on the partial saved artifacts (task-level 50/70 dev/holdout split, source quality estimated on dev only) found no conversion: on the 70-task holdout the best dev source solves 1/70, best-source attempt-1 plus cross-source attempt-2 also solves 1/70, while the union oracle is 8/70. Cross-source diversification does not convert the observed pool headroom on these partial artifacts. This is an appendix note only; it is not part of the pre-registered study, the artifacts are partial eval-mode runs rather than leaderboard reruns, and source quality was estimated on the dev split only.